

Assignment 2 di Programmazione Concorrente e Distribuita 2025/2026

Mattia Ronchi, matr. 0001236997

Samorì Andrea matr. 0001235969

Andrea Monaco matr. 0001225150

Analisi del problema

L'assignment ha l'obiettivo di realizzare una libreria chiamata `FSStatLib` che fornisce un metodo asincrono chiamato `getFSReport`. Questo metodo permette di ottenere alcune statistiche sulla dimensione dei file in una cartella `D` (inclusendo le sottocartelle ricorsivamente). Il report `R` fornito come output del metodo include:

- numero totale di file appartenenti a `D` (inclusendo le sottocartelle ricorsivamente)
- la distribuzione delle dimensioni dei file. Data una dimensione `MaxFS` e un numero di bande `NB`, il metodo computa, per ogni banda, il numero di file che vi appartengono.

Aspetti rilevanti per la concorrenza

L'aspetto di concorrenza per noi più interessante è la navigazione delle sottocartelle.

Abbiamo, come da richiesta, sviluppato tre soluzioni usando tre differenti approcci di sviluppo:

- programmazione asincrona basata su event-loop
- programmazione reattiva usando `Rx`
- thread virtuali

Ora andremo nel dettaglio dei vari approcci.

Design della soluzione

Event-loop

Per sviluppare una soluzione basata su event-loop, abbiamo utilizzato il framework `Vert.x`, in particolare ci siamo avvalsi di `Vertx.fileSystem`, che permette di avere un event-loop apposito per operazioni su file e cartelle.

La funzione che svolge tutto il lavoro è `calculateBands`, che ritorna un `Future<ReportResult>`. Questa funzione controlla la natura del path preso in input.

Se è un file, calcola la sua dimensione e lo assegna alla sua banda. Fatto ciò, ritorna una `Future.succeededFuture` con un `ReportResult` vuoto, se non per il file appena aggiunto.

Se abbiamo una cartella, per ogni path all'interno, viene chiamata ricorsivamente `calculateBands`. Finiti tutti questi path interni, vengono aggregati tutti i risultati in una unica `Future<ReportResult>`.

Programmazione reattiva

Per sviluppare una soluzione basata sulla programmazione reattiva, abbiamo utilizzato il framework `RxJava`.

Creiamo in primo luogo un scheduler, che spartirà le varie computazioni a diversi thread.

Tramite la funzione `walkFiles`, viene ritornato un `Observable<Long>` che rappresenta la dimensione di ogni file. Queste dimensioni vengono aggregate per banda nella funzione `getFSReport` serialmente.

walkFiles viene richiamata ricorsivamente se incontra una cartella, e questa chiamata avviene tramite subscribeOn(scheduler). Questo permette la parallelizzazione dell'esplorazione delle sottocartelle.

```
return Observable.fromArray(children)
    .flatMap(x -> walkFiles(x).subscribeOn(this.scheduler));
```

Thread virtuali

In questa soluzione abbiamo delegato la computazione a dei thread virtuali, creati tramite un ExecutorService.

```
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
```

Qui dobbiamo assicurarci noi che il risultato venga restituito solo quando tutti i thread virtuali creati abbiamo finito la computazione. Per fare questo abbiamo utilizzato un phaser: una barriera riutilizzabile e più flessibile in quanto permette di aspettare un numero dinamico (e non noto alla creazione) di parties.

```
public Future<Report> getFSReport(String directory, long maxFS, int bands) {
    ...
    this.phaser = new Phaser(1); // <- initial number of parties

    submit(() -> calculateSize(path));

    return executor.submit(() -> {
        phaser.awaitAdvance(phaser.arrive());
        return report;
    });
}

private void submit(Runnable action) {
    phaser.register();
    executor.submit(() -> {
        try {
            action.run();
        } finally {
            phaser.arriveAndDeregister();
        }
    });
}
```

Tramite il metodo calculateSize(Path path), aggiorniamo il monitor condiviso Report, che tiene traccia del numero totale di file e delle varie bande. Questo metodo viene chiamato ricorsivamente sulle sottocartelle se il path in input è una cartella

Punto opzionale: interfaccia utente interattiva

Abbiamo progettato l'interfaccia per la soluzione che sfrutta i thread virtuali. Viene mostrato in tempo reale l'aggiornamento del totale del numero di file esplorati e il numero di file per ogni banda. Tramite il pulsante Stop è possibile fermare la computazione anche prima del suo naturale completamento. Poi, possiamo iniziare una nuova computazione (volendo anche su nuovi parametri acquisiti tramite i TextField) utilizzando il pulsante Start. Gli asterischi danno un'idea grafica della distribuzione della dimensione dei file nella cartella.

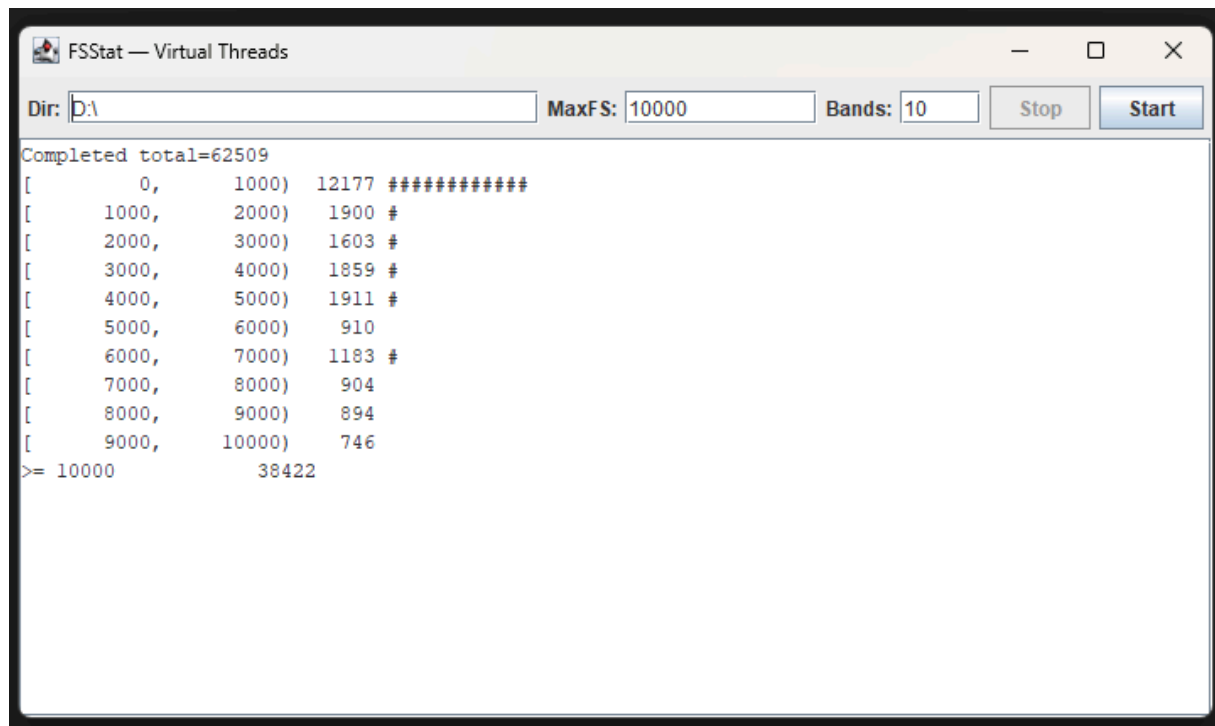


Figure 1: Interfaccia grafica interattiva.